

Codelab: Migrating Existing UI to Jetpack Compose

Repository : <https://code.nbs.dev/abrar/composemigration>

Introduction

In modern Android development, **Jetpack Compose** is the recommended way to build UI. This codelab guides you through the process of migrating a core UI component—specifically, a `CardView` within a `Fragment` that observes data from a `ViewModel`—from the classic Android XML/View system to a pure, declarative Compose structure. We will utilize the `content` extension function for the cleanest possible integration, while maintaining the existing `Fragment` to facilitate an incremental, interoperability-focused migration.

What You'll Do

You will migrate a complete UI feature, step-by-step, including:

1. **Translating XML Resources:** Converting `colors.xml` and `styles.xml` (Text Appearances) into Compose's `ColorScheme` and `Typography`.
2. **Creating a Composable Component:** Replacing the existing XML layout (`CardView`, `LinearLayouts`) with a `@Composable` function for both component and screen level UI.
3. **Implementing Interoperability:** Using the concise `content` extension function in the `Fragment`'s `onCreateView` to host the Compose UI and safely observe `StateFlow` with Lifecycle Aware fashion, eliminating the need for `ComposeView` boilerplate and avoiding leaks.

What You'll Learn

- The direct translation of XML layout components (`CardView`, `LinearLayout`, `TextView`) to Compose equivalents (`Card`, `Row`, `Column`, `Text`).
- How to correctly map XML-based styling to Compose theming.
- The recommended pattern for observing Kotlin `StateFlow` in a Composable using `collectAsStateWithLifecycle`.

- How to leverage the modern `androidx.fragment.compose.content` extension for smooth and concise **Fragment-to-Compose** interoperability.

Step 1: Defining the Compose Theme Foundation

The foundation of a good Compose migration is a robust theme system. You must translate your XML resources into the corresponding Compose classes (`Color`, `ColorScheme`, `Typography`).

1.1 Colors and Color Scheme

Since we only use a single theme, we only need to use a single color scheme. We can convert the custom colors and map them to the `DarkColorScheme`. Note how `colorPrimaryContainer` and `colorOnPrimaryContainer` are mapped.

```
<color name="colorPrimary">#41054B</color>
<color name="colorBackground">#1E0021</color>
<color name="colorContainer">#28002B</color>
```

```
<style name="Theme.Composemigration" parent="Theme.Material3.DayNight.NoActionBar">
    <item name="colorPrimary">@color/colorPrimary</item>
    <item name="android:colorBackground">@color/colorBackground</item>
    <item name="colorPrimaryContainer">@color/colorContainer</item>
    <item name="colorOnPrimaryContainer">@color/white</item>
</style>
```

```
5  val Purple80 = Color( color = 0xFF41054B)
6  val DarkPurple = Color( color = 0xFF1E0021)
7  val MediumPurple = Color( color = 0xFF28002B)
```

```
internal val DarkColorScheme = darkColorScheme(
    primary = Purple80,
    background = DarkPurple, // Maps to `android:colorBackground`
    surface = DarkPurple, // Often set to the same as background in simple dark schemes
    primaryContainer = MediumPurple,
    onPrimaryContainer = Color.White,
)
```

1.2 Custom Typography

We define the custom text styles, ensuring the correct font size and weight based on original XML `TextAppearance` definitions. By default, we can not customize the style name like what we did in XML.

```
<style name="TextAppearance.App.Header" parent="TextAppearance.Material3.TitleMedium">
    <item name="android:textSize">16sp</item>
    <item name="android:textStyle">bold</item>
    <item name="android:textColor">?attr/colorOnPrimaryContainer</item>
</style>

<style name="TextAppearance.App.ListItemTitle" parent="TextAppearance.Material3.BodyLarge">
    <item name="android:textSize">14sp</item>
    <item name="android:textStyle">bold</item>
    <item name="android:textColor">?attr/colorOnPrimaryContainer</item>
</style>

<style name="TextAppearance.App.Caption" parent="TextAppearance.Material3.LabelSmall">
    <item name="android:textSize">11sp</item>
    <item name="android:textStyle">normal</item>
    <item name="android:textColor">?attr/colorOnPrimaryContainer</item>
</style>
```

```
private val typographyColor = DarkColorScheme.onPrimaryContainer
```

1 Usage

```
val AppTypography = Typography(
    titleMedium = TextStyle(
        fontSize = 16.sp,
        fontWeight = FontWeight.Bold,
        color = typographyColor,
    ),
    bodyLarge = TextStyle(
        fontSize = 14.sp,
        fontWeight = FontWeight.Bold,
        color = typographyColor,
    ),
    labelSmall = TextStyle(
        fontSize = 11.sp,
        fontWeight = FontWeight.Normal,
        color = typographyColor,
    )
)
```

1.3 Define Theme Wrapper Composable

We define `AppTheme` composables to act as Theming wrapper for every Composable we passed in.

```
@Composable
fun AppTheme(content: @Composable () -> Unit, stable) {
    MaterialTheme(
        colorScheme = DarkColorScheme,
        typography = AppTypography,
        content = content,
    )
}
```

Step 2: Creating Composable Item & List

This step replaces the XML item layout and the RecyclerView setup with two Compose functions: `TopScorerItem` and `TopScorerList`.

2.1 The `TopScorerItem` Composable (The List Item)

The `TopScorerItem` Composable directly replaces your XML `CardView` item layout. It accepts the necessary data and click callback.

```
@Composable
fun TopScorerItem(
    scorer: TopScorer,
    rank: Int,
    onClick: (Int) -> Unit
) {
    Card(
        modifier = Modifier
```

```
        .fillMaxWidth()

        .clickable { onClick(scorer.id) },

        shape = RoundedCornerShape(12.dp),

        colors = CardDefaults.cardColors(

            containerColor =
MaterialTheme.colorScheme.primaryContainer

        ),

        elevation = CardDefaults.cardElevation(defaultElevation =
0.dp)
    ) {

        Row(

            modifier = Modifier

                .fillMaxWidth()

                .padding(16.dp),

            verticalAlignment = Alignment.CenterVertically
        ) {

            Text(

                text = rank.toString(),

                modifier = Modifier.padding(end = 16.dp),

                style = MaterialTheme.typography.bodyLarge,

            )

            AsyncImage(

                model = scorer.imageUrl,

                contentDescription = "Player Photo",

                contentScale = ContentScale.FillHeight,
```

```

        modifier = Modifier

            .size(40.dp)

            .padding(end = 16.dp),

    )

    Column(

        modifier = Modifier

            .weight(1f)

            .padding(end = 8.dp),

    ) {

        Text(

            text = scorer.name,

            style = MaterialTheme.typography.bodyLarge,

        )

        Row(

            modifier = Modifier.padding(top = 4.dp),

            verticalAlignment = Alignment.CenterVertically,

        ) {

            AsyncImage(

                model = scorer.club.logoUrl,

                contentDescription = "Club Logo",

                contentScale = ContentScale.Fit,

                modifier = Modifier

                    .size(16.dp)

```

```

                .padding(end = 8.dp),

            )

            Text(

                text = scorer.club.name,

                style = MaterialTheme.typography.labelSmall,

            )

        }

    }

    Text(

        text = scorer.goal.toString(),

        style = MaterialTheme.typography.titleMedium,

        modifier = Modifier.align(Alignment.CenterVertically),

    )

}

}

}

```

2.2 The **TopScorersList** Composable (The List Host)

This function uses **LazyColumn** to iterate over the list data. To simulate non-clipped bottom padding in RecyclerView, we passed navigation bar padding into **LazyColumn** contentPadding.

```

@Composable

fun TopScorersList(

```

```
scorers: List<TopScorer>,

navigationBarPadding: PaddingValues,

onClick: (Int) -> Unit,

modifier: Modifier = Modifier,
) {

    LazyColumn(

        modifier = modifier.fillMaxSize(),

        verticalArrangement = Arrangement.spacedBy(8.dp),

        contentPadding = PaddingValues(bottom =
navigationBarPadding.calculateBottomPadding()),

    ) {

        itemsIndexed(scorers) { index, scorer ->

            val rank = index + 1

            TopScorerItem(

                scorer = scorer,

                rank = rank,

                onClick = onClick,

            )

        }

    }

}
```


Step 3: Migrating MainActivity

The `MainActivity` is simplified by replacing `setContentview(view)` with `setContent`, using the `TopScorerScreen` as the root Composable.

3.1 The Main Screen Composable (`TopScorerScreen`)

This new Composable defines the complete visual structure of the main screen, including the `Scaffold`, which typically replaces the parent Activity's layout logic. It hosts the list wrapper, `TopScorerList`. The list data is managed by local state and loaded once using `LaunchedEffect` when the screen is first shown.

```
@Composable

fun TopScorerScreen(
    onScorerClick: (Int) -> Unit
) {
    var scorers by remember { mutableStateOf(emptyList<TopScorer>()) }

    LaunchedEffect(Unit) {
        scorers = topScorers
    }

    Scaffold(
        modifier = Modifier
            .fillMaxSize()
    ) { paddingValues ->
        val navigationBarPadding =
            WindowInsets.navigationBars.asPaddingValues()

        Column(
```

```
        modifier = Modifier

            .fillMaxSize()

            .padding(top = paddingValues.calculateTopPadding())
    ) {

        Text(

            text = "Top Scorer",

            modifier = Modifier.padding(16.dp),

            style = MaterialTheme.typography.titleMedium,

        )

        TopScorersList(

            scorers = scorers,

            navigationBarPadding = navigationBarPadding,

            onClick = onScorerClick,

            modifier = Modifier

                .fillMaxWidth()

                .weight(1f)

                .padding(horizontal = 16.dp),

        )

    }

}

}
```

3.2 Update MainActivity

We replace the `MainActivity`'s XML layout reference (`setContentView(binding.root)`) with Compose's `setContent` function, and host the new `TopScorerScreen` component. This means we can also remove the binding reference.

```
setContent {  
    AppTheme {  
        TopScorerScreen(  
            onScorerClick = { scorerId ->  
                startActivity(  
                    Intent(this, DetailActivity::class.java)  
                        .putExtra(DetailActivity.EXTRA_SCORER_ID,  
scorerId)  
                )  
            }  
        )  
    }  
}
```

Step 4: Fragment Interoperability (The Detail Screen)

This final step involves migrating the player card view inside a **Fragment**. We first create the Composable that replaces the Fragment's XML layout, and then use it using the `content` extension function.

4.1 Migrate Layout to Composable (**PlayerCard**)

This Composable replaces the original `fragment_player_card.xml` layout. It is responsible for rendering the player card UI.

```
@Composable
fun PlayerCard(
    scorer: TopScorer,
    bio: DetailedScorerBio,
) {
    val club = scorer.club

    val cardBackgroundColor = remember(club.primaryColorHex) {
        Color(club.primaryColorHex.toColorInt())
    }

    Card(
        modifier = Modifier.fillMaxWidth(),
        shape =
            androidx.compose.foundation.shape.RoundedCornerShape(16.dp),
        elevation = CardDefaults.cardElevation(0.dp),
        colors = CardDefaults.cardColors(
            containerColor = cardBackgroundColor,
        ),
    )
}
```

```
) {  
  
    Row(  
  
        modifier = Modifier  
            .fillMaxWidth()  
            .padding(16.dp),  
        verticalAlignment = Alignment.CenterVertically,  
    ) {  
  
        AsyncImage(  
            model = scorer.imageUrl,  
            contentDescription = "Player Photo",  
            contentScale = ContentScale.Inside,  
            modifier = Modifier  
                .width(80.dp)  
                .height(100.dp),  
        )  
  
        Column(  
            modifier = Modifier  
                .weight(1f)  
                .align(Alignment.CenterVertically)  
                .padding(start = 16.dp),  
        ) {  
  
            Text(  

```

```
        text = scorer.name,

        style = MaterialTheme.typography.titleMedium,

    )

    Row(

        modifier = Modifier.padding(top = 8.dp),

        verticalAlignment = Alignment.CenterVertically,

    ) {

        AsyncImage(

            model = club.logoUrl,

            contentDescription = "Club Logo",

            contentScale = ContentScale.Fit,

            modifier = Modifier

                .size(16.dp)

                .padding(end = 4.dp),

        )

        Text(

            text = club.name,

            style = MaterialTheme.typography.labelSmall,

        )

    }

    Row(

        modifier = Modifier.padding(top = 2.dp),
```

```
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Text(
            text = bio.position,
            style = MaterialTheme.typography.labelSmall,
        )

        Text(
            text = ".",
            style = MaterialTheme.typography.labelSmall,
            modifier = Modifier
                .padding(start = 6.dp)
                .padding(end = 6.dp),
        )

        Text(
            text = bio.shirtNumber.toString(),
            style = MaterialTheme.typography.labelSmall,
        )
    }
}
}
}
```

4.2 Integrate Composable into Fragment (**PlayerCardFragment**)

We update the **PlayerCardFragment**'s **onCreateView** method to use the **content** extension function, hosting the new **PlayerCardContainer** Composable. This completely replaces the need for XML inflation (**inflater.inflate(...)**). But before that, we need to add fragment-compose dependencies first.

```
libs.versions.toml

androidx-fragment-compose = { group = "androidx.fragment", name =
"fragment-compose", version.ref = "fragment" }

build.gradle:

implementation(libs.androidx.fragment.compose)
```

```
PlayerCardFragment:

override fun onCreateView(

    inflater: LayoutInflater,

    container: ViewGroup?,

    savedInstanceState: Bundle?,

): View = content {

    AppTheme {

        val details by
viewModel.scorerDetail.collectAsStateWithLifecycle(initialValue =
null)

        details?.let { (topScorer, detailedBio) ->

            PlayerCard(

                scorer = topScorer,

                bio = detailedBio,
```



```
    )  
    }  
}  
}
```

Summary

You have successfully completed the migration of your entire UI layer, starting with the theme, moving to the list item, then the list host, the Activity entry point, and finally the detail fragment interoperability. Your new architecture is clean, declarative, and ready for future Compose development!